The Verlet Algorithm

Prof. Dr. Ademir João Camargo

1 VERLET ALGORITHM....1
2 VELOCITY VERLET....3
3 LEAPFROG INTEGRATION....4
4 CHOOSING THE TIME STEP....5
5 NUMERICAL EVALUATION OF FORCES....6

1 Verlet Algorithm

In a molecular dynamics (MD) simulation, including Car–Parrinello simulations, we follow atomic motion by numerically solving Newton's equations of motion. Time is divided into small, equal steps of length δt, called the *integration step* or *time step*. Starting from the positions (and velocities) of the atoms at time t, an integration algorithm tells us how to compute them at the next time, $t + \delta t$. Repeating this procedure step after step produces a *trajectory*.

The Verlet algorithm, introduced by Loup Verlet in 1967, is one of the simplest and most widely used integration algorithms. It is derived from two Taylor expansions of the position $\mathbf{r}(t)$ of an atom: one forward in time, to $t + \delta t$, and one backward in time, to $t - \delta t$:

$$\mathbf{r}(t + \delta t) = \mathbf{r}(t) + \dot{\mathbf{r}}(t)\,\delta t + \frac{1}{2}\ddot{\mathbf{r}}(t)\,(\delta t)^2 + \frac{1}{3!}\dddot{\mathbf{r}}(t)\,(\delta t)^3 + O((\delta t)^4), \quad (1)$$

$$\mathbf{r}(t - \delta t) = \mathbf{r}(t) - \dot{\mathbf{r}}(t)\,\delta t + \frac{1}{2}\ddot{\mathbf{r}}(t)\,(\delta t)^2 - \frac{1}{3!}\dddot{\mathbf{r}}(t)\,(\delta t)^3 + O((\delta t)^4). \quad (2)$$

Here, $\mathbf{r}$ is the position vector of the atom, t is the time, and each dot denotes one time derivative: $\dot{\mathbf{r}} = \mathbf{v}$ is the velocity, $\ddot{\mathbf{r}} = \mathbf{a}$ is the acceleration, and $\dddot{\mathbf{r}}$ is the time derivative of the acceleration. The symbol $O((\delta t)^4)$ collects all the terms of fourth and higher order in δt, which we neglect. Notice that the two expansions differ only in the signs of the odd-power terms.

Adding (1) and (2), the terms in δt and $(\delta t)^3$ cancel, and we obtain

$$\mathbf{r}(t + \delta t) + \mathbf{r}(t - \delta t) = 2\mathbf{r}(t) + \ddot{\mathbf{r}}(t)\,(\delta t)^2 + O((\delta t)^4). \quad (3)$$

The acceleration is given by Newton's second law,

$$\ddot{\mathbf{r}}(t) = \mathbf{a}(t) = \frac{\mathbf{F}(t)}{m}, \quad (4)$$

where $\mathbf{F}(t)$ is the force acting on the atom at time t and m is its mass. Solving (3) for $\mathbf{r}(t + \delta t)$, substituting (4), and dropping the $O((\delta t)^4)$ terms gives the *position Verlet* update:

$$\mathbf{r}(t+\delta t) \approx 2\mathbf{r}(t) - \mathbf{r}(t-\delta t) + \frac{\mathbf{F}(t)}{m}\,(\delta t)^2. \tag{5}$$

To compute the new position $\mathbf{r}(t+\delta t)$, we therefore need the current position $\mathbf{r}(t)$, the previous position $\mathbf{r}(t-\delta t)$, and the current force $\mathbf{F}(t)$; the velocities are not needed. Because two earlier positions are required, the very first step must be done differently: a common choice is to compute $\mathbf{r}(\delta t)$ from the initial position and velocity using (1) truncated after the $(\delta t)^2$ term.

Because the third-order terms cancel, the error made in a *single* step of (5) is of order $(\delta t)^4$ (the *local error*). To simulate a fixed time interval T, however, we need $T/\delta t$ steps, and the local errors accumulate. As a result, the error in the positions after a fixed simulation time (the *global error*) is of order $(\delta t)^2$. For this reason, the Verlet algorithm is called a *second-order* method: halving the time step reduces the global error by about a factor of four.

For a conservative system, the force is minus the gradient of the potential energy $V(\mathbf{r})$, which is a scalar function of the position:

$$\mathbf{F}(\mathbf{r}) = -\nabla V(\mathbf{r}) = -\left(\frac{\partial V}{\partial x}, \frac{\partial V}{\partial y}, \frac{\partial V}{\partial z}\right). \tag{6}$$

In a system of N atoms, V depends on all the positions $\mathbf{r}_1, \ldots, \mathbf{r}_N$, and the force on atom i is obtained by differentiating V with respect to the coordinates of that atom, $\mathbf{F}_i = -\nabla_i V$. Evaluating the forces is usually the most expensive part of each step.

Although (5) does not use the velocities, we need them to compute the kinetic energy and the temperature. We can estimate them from the positions. Subtracting (2) from (1), the even-power terms cancel:

$$\mathbf{r}(t+\delta t) - \mathbf{r}(t-\delta t) = 2\mathbf{v}(t)\,\delta t + O((\delta t)^3). \tag{7}$$

Dividing by $2\delta t$ gives the *central-difference* estimate of the velocity,

$$\mathbf{v}(t) = \frac{\mathbf{r}(t+\delta t) - \mathbf{r}(t-\delta t)}{2\,\delta t} + O((\delta t)^2). \tag{8}$$

Note that dividing by δt lowers the order of the error by one: the remainder in (7) is of order $(\delta t)^3$, but the error in the velocity is of order $(\delta t)^2$. Note also that $\mathbf{v}(t)$ becomes available only after $\mathbf{r}(t+\delta t)$ has been computed, that is, one step late.

From the velocities, we obtain the kinetic energy $K = \sum_i \frac{1}{2} m_i \mathrm{v}_i^2$ and, by the equipartition theorem, the instantaneous temperature

$$T_{\text{inst}} = \frac{2K}{N_f\, k_B}, \tag{9}$$

where k_B is the Boltzmann constant and N_f is the number of unconstrained degrees of freedom ($N_f = 3N$ minus the number of constraints; for example, $N_f = 3N - 3$ when the motion of the center of mass is removed).

2 Velocity Verlet

In the position Verlet algorithm, the velocity at time t is obtained only after the position at $t + \delta t$ is known. The *velocity Verlet* algorithm (Swope, Andersen, Berens, and Wilson, 1982) instead advances positions and velocities together, so that both are known at the same time t.

The position update is obtained by truncating (1) after the $(\delta t)^2$ term and writing $\dot{\mathbf{r}} = \mathbf{v}$ and $\ddot{\mathbf{r}} = \mathbf{a}$:

$$\mathbf{r}(t + \delta t) = \mathbf{r}(t) + \mathbf{v}(t)\,\delta t + \frac{1}{2}\mathbf{a}(t)\,(\delta t)^2. \tag{10}$$

For the velocity, we write a separate Taylor expansion of $\mathbf{v}(t)$ (remember that $\dot{\mathbf{v}} = \mathbf{a}$):

$$\mathbf{v}(t + \delta t) = \mathbf{v}(t) + \mathbf{a}(t)\,\delta t + \frac{1}{2}\dot{\mathbf{a}}(t)\,(\delta t)^2 + O((\delta t)^3). \tag{11}$$

The derivative of the acceleration, $\dot{\mathbf{a}}(t)$, is not known, so we approximate it by a forward finite difference:

$$\dot{\mathbf{a}}(t) = \frac{\mathbf{a}(t + \delta t) - \mathbf{a}(t)}{\delta t} + O(\delta t). \tag{12}$$

Substituting (12) into (11) and simplifying step by step:

$$\begin{aligned}
\mathbf{v}(t + \delta t) &= \mathbf{v}(t) + \mathbf{a}(t)\,\delta t + \frac{1}{2}\,\frac{\mathbf{a}(t + \delta t) - \mathbf{a}(t)}{\delta t}\,(\delta t)^2 + O((\delta t)^3) \\
&= \mathbf{v}(t) + \mathbf{a}(t)\,\delta t + \frac{1}{2}[\mathbf{a}(t + \delta t) - \mathbf{a}(t)]\delta t + O((\delta t)^3) \\
&= \mathbf{v}(t) + \frac{1}{2}[\mathbf{a}(t) + \mathbf{a}(t + \delta t)]\delta t + O((\delta t)^3).
\end{aligned}$$

The $O(\delta t)$ error of (12) is multiplied by $\frac{1}{2}(\delta t)^2$, so it contributes only an $O((\delta t)^3)$ term, which is of the same order as the terms already neglected in (11). Finally, using $\mathbf{a} = \mathbf{F}/m$, the two velocity Verlet equations are

$$\mathbf{r}(t + \delta t) = \mathbf{r}(t) + \mathbf{v}(t)\,\delta t + \frac{\mathbf{F}(t)}{2m}\,(\delta t)^2, \tag{13}$$

$$\mathbf{v}(t + \delta t) = \mathbf{v}(t) + \frac{\mathbf{F}(t) + \mathbf{F}(t + \delta t)}{2m}\,\delta t. \tag{14}$$

Equation (14) uses the forces at both the old and the new positions. In practice, one step of the algorithm is carried out as follows:

1) Compute the new positions $\mathbf{r}(t + \delta t)$ with (13).
2) Evaluate the new forces $\mathbf{F}(t + \delta t)$ at the new positions.
3) Compute the new velocities $\mathbf{v}(t + \delta t)$ with (14).

The force $\mathbf{F}(t + \delta t)$ computed in step 2 is reused as $\mathbf{F}(t)$ in the next step, so only one force evaluation is needed per step.

Velocity Verlet produces the same positions as (5). To see this, write (13) for the step from $t - \delta t$ to t,

$$\mathbf{r}(t) = \mathbf{r}(t - \delta t) + \mathbf{v}(t - \delta t)\,\delta t + \frac{\mathbf{F}(t - \delta t)}{2m}\,(\delta t)^2,$$

and subtract it from (13):

$$\mathbf{r}(t + \delta t) - 2\mathbf{r}(t) + \mathbf{r}(t - \delta t) = [\mathbf{v}(t) - \mathbf{v}(t - \delta t)]\delta t + \frac{\mathbf{F}(t) - \mathbf{F}(t - \delta t)}{2m}\,(\delta t)^2.$$

According to (14), $\mathbf{v}(t) - \mathbf{v}(t - \delta t) = [\mathbf{F}(t - \delta t) + \mathbf{F}(t)]\delta t/(2m)$. Substituting this result, the terms in $\mathbf{F}(t - \delta t)$ cancel, and we recover $\mathbf{r}(t + \delta t) = 2\mathbf{r}(t) - \mathbf{r}(t - \delta t) + \mathbf{F}(t)\,(\delta t)^2/m$, which is (5).

Like position Verlet, velocity Verlet is a second-order, time-reversible method. Its total energy is not conserved exactly at every step; instead, for a sufficiently small time step it fluctuates around a constant value without drifting systematically, even over long simulations. (If the forces depend on the velocities, as in some constant-temperature or constant-pressure schemes, (14) must be modified or solved iteratively, because $\mathbf{F}(t + \delta t)$ then depends on $\mathbf{v}(t + \delta t)$.)

3 Leapfrog Integration

In the *leapfrog* algorithm, which has been implemented in molecular-simulation programs such as HyperChem, the positions are computed at the times $t, t + \delta t, t + 2\delta t, \ldots$, while the velocities are computed at the intermediate "half-step" times $t - \delta t/2, t + \delta t/2, \ldots$.

To derive the position update, we expand the position about the midpoint $t + \delta t/2$, forward and backward by half a step:

$$\mathbf{r}(t + \delta t) = \mathbf{r}(t + \delta t/2) + \mathbf{v}(t + \delta t/2)\frac{\delta t}{2} + \frac{1}{2}\mathbf{a}(t + \delta t/2)\left(\frac{\delta t}{2}\right)^2 + O((\delta t)^3), \tag{15}$$

$$\mathbf{r}(t) = \mathbf{r}(t + \delta t/2) - \mathbf{v}(t + \delta t/2)\frac{\delta t}{2} + \frac{1}{2}\mathbf{a}(t + \delta t/2)\left(\frac{\delta t}{2}\right)^2 + O((\delta t)^3). \tag{16}$$

Subtracting (16) from (15), the zeroth- and second-order terms cancel:

$$\mathbf{r}(t + \delta t) - \mathbf{r}(t) = \mathbf{v}(t + \delta t/2)\delta t + O((\delta t)^3),$$

which gives the leapfrog position update

$$\mathbf{r}(t + \delta t) = \mathbf{r}(t) + \mathbf{v}(t + \delta t/2)\delta t. \tag{17}$$

The velocity update is derived in the same way, now expanding the velocity about time t, forward and backward by half a step:

$$\mathbf{v}(t+\delta t/2) = \mathbf{v}(t) + \mathbf{a}(t)\,\frac{\delta t}{2} + \frac{1}{2}\dot{\mathbf{a}}(t)\left(\frac{\delta t}{2}\right)^2 + O((\delta t)^3), \tag{18}$$

$$\mathbf{v}(t-\delta t/2) = \mathbf{v}(t) - \mathbf{a}(t)\,\frac{\delta t}{2} + \frac{1}{2}\dot{\mathbf{a}}(t)\left(\frac{\delta t}{2}\right)^2 + O((\delta t)^3). \tag{19}$$

Subtracting (19) from (18) gives $\mathbf{v}(t+\delta t/2) - \mathbf{v}(t-\delta t/2) = \mathbf{a}(t)\,\delta t + O((\delta t)^3)$, and with $\mathbf{a} = \mathbf{F}/m$,

$$\mathbf{v}(t+\delta t/2) = \mathbf{v}(t-\delta t/2) + \frac{\mathbf{F}(t)}{m}\,\delta t. \tag{20}$$

Equations (17) and (20) define the leapfrog algorithm. In each step, we first compute the force $\mathbf{F}(t)$ from the positions $\mathbf{r}(t)$, then advance the velocities to $t + \delta t/2$ with (20), and finally advance the positions to $t + \delta t$ with (17). Positions and velocities "leap over" each other on staggered time grids, which explains the name of the method.

If the velocity is needed at the same time as the positions (for example, to compute the kinetic energy at time t), it can be obtained by adding (18) and (19):

$$\mathbf{v}(t) = \frac{1}{2}[\mathbf{v}(t-\delta t/2) + \mathbf{v}(t+\delta t/2)] + O((\delta t)^2). \tag{21}$$

Eliminating the velocities from (17) and (20) again leads to (5), so leapfrog, position Verlet, and velocity Verlet generate the same trajectory; they differ only in how and when the velocities are computed.

4 Choosing the Time Step

A larger time step means fewer force evaluations per unit of simulated time, so the simulation is cheaper. However, the time step cannot be too large: if it is, the trajectory becomes inaccurate and, eventually, the integration becomes unstable and the energy "explodes." A useful rule of thumb is that δt should be roughly one-tenth or less of the period of the fastest motion in the system. Bond vibrations involving hydrogen atoms are usually the fastest motions, which is why time steps of about 0.5 to 1 fs are common in simulations that include them.

Higher-order integration methods allow larger steps for the same accuracy over short times, but they require more force evaluations, more derivatives, or more storage per step. Moreover, a higher order does not guarantee better long-term energy conservation. Verlet-type methods are *symplectic*, which means they preserve important geometric properties of Hamiltonian dynamics; as a consequence, for a sufficiently small, fixed time step, their energy error remains bounded over very long simulations. This favorable behavior can be lost if the forces are inaccurate, constraints are not handled carefully, or the time step changes during the simulation.

Finally, molecular systems are *chaotic*: two trajectories that start from almost identical initial conditions diverge from each other exponentially in time. Small integration and round-off errors therefore make any numerical trajectory depart, after some time, from the exact trajectory with the

same initial conditions. This does not prevent us from computing reliable *average* properties (such as average energy, pressure, or structure), but we must check their convergence by running the simulation long enough and sampling adequately.

In summary, the typical properties of Verlet-type algorithms are:

1) Simplicity and low computational cost: only one force evaluation per step.
2) Second-order accuracy: the global error is proportional to $(\delta t)^2$.
3) Modest memory requirements.
4) Time reversibility.
5) Good long-term energy conservation: for a suitable time step, the total energy fluctuates around a constant value without a systematic drift.

5 Numerical Evaluation of Forces

When the analytical gradient of the energy is not available (as is the case for some quantum-chemical methods), the forces can be estimated from energy differences using finite differences. In a system with many atoms, we displace one Cartesian coordinate at a time by a small amount h, while all the other coordinates are kept fixed. It is therefore enough to consider the energy as a function $E(x)$ of a single coordinate x. Its Taylor expansions for a displacement $+h$ and $-h$ are

$$E(x+h) = E(x) + E'(x)\,h + \frac{1}{2}E''(x)\,h^2 + \frac{1}{3!}E'''(x)\,h^3 + O(h^4), \tag{22}$$

$$E(x-h) = E(x) - E'(x)\,h + \frac{1}{2}E''(x)\,h^2 - \frac{1}{3!}E'''(x)\,h^3 + O(h^4). \tag{23}$$

Subtracting (23) from (22), the even-power terms cancel:

$$E(x+h) - E(x-h) = 2E'(x)\,h + \frac{1}{3}E'''(x)\,h^3 + O(h^4). \tag{24}$$

Dividing by $2h$ gives the central-difference formula for the derivative,

$$E'(x) = \frac{E(x+h) - E(x-h)}{2h} + O(h^2). \tag{25}$$

From (24), the leading error term is $E'''(x)\,h^2/6$; halving h therefore reduces the error by about a factor of four. Since the force component along x is $F_x = -\,\partial E/\,\partial x$, the force component on atom α along its coordinate x_α is

$$F_{\alpha,x} = -\frac{E(\dots, x_\alpha + h, \dots) - E(\dots, x_\alpha - h, \dots)}{2h} + O(h^2), \tag{26}$$

where the dots indicate that all the other coordinates are held fixed. The components $F_{\alpha,y}$ and $F_{\alpha,z}$ are obtained in the same way. Because each force component requires two energy calculations, the complete force vector for N atoms require $6N$ energy calculations, which makes this approach expensive for large systems.

The displacement h must be chosen with care. If h is too large, the truncation error $O(h^2)$ becomes large; if h is too small, the difference $E(x+h) - E(x-h)$ becomes comparable to the

numerical noise in the computed energies (for example, from incomplete convergence of a self-consistent-field calculation), and round-off errors dominate. In practice, a suitable value of h is found by testing.